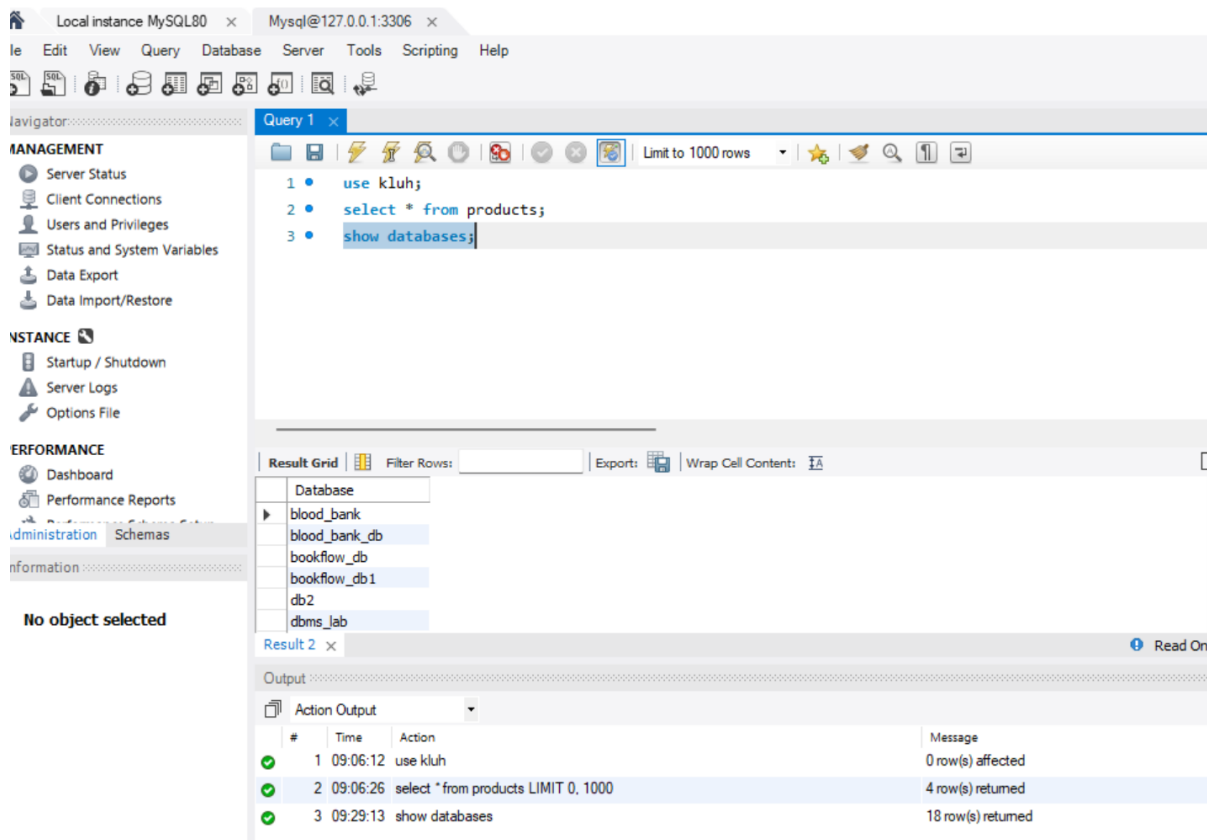


DBSE AND DBD

Database Schema & Queries (MySQL)

Create Database

CREATE DATABASE kluh;



The screenshot displays the MySQL Workbench interface. The left sidebar shows the 'MANAGEMENT' and 'INSTANCE' sections. The main window shows a query window with the following SQL commands:

```
1 • use kluh;  
2 • select * from products;  
3 • show databases;
```

The 'Result Grid' shows the output of the 'show databases;' query, listing the following databases:

Database
blood_bank
blood_bank_db
bookflow_db
bookflow_db1
db2
dbms_lab

The 'Output' pane shows the execution results for each query step:

#	Time	Action	Message
✓ 1	09:06:12	use kluh	0 row(s) affected
✓ 2	09:06:26	select * from products LIMIT 0, 1000	4 row(s) returned
✓ 3	09:29:13	show databases	18 row(s) returned

Use Database

USE kluh;

MySQL Workbench

Local instance MySQL80 x Local instance MySQL80 x TestSQL x

File Edit View Query Database Server Tools Scripting Help

Navigator

MANAGEMENT

- Server Status
- Client Connections
- Users and Privileges
- Status and System Variables
- Data Export
- Data Import/Restore

INSTANCE Administration Schemas

Information

No object selected

SQL File 5 SQL File 6* SQL File 7

Limit to 1000 rows

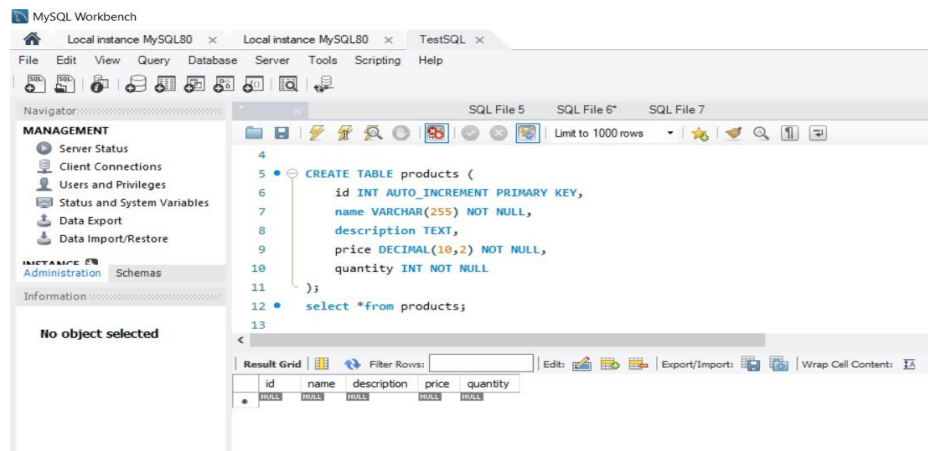
```
1 • CREATE DATABASE kluh;
2 • show databases;
3 • USE kluh;
```

Output

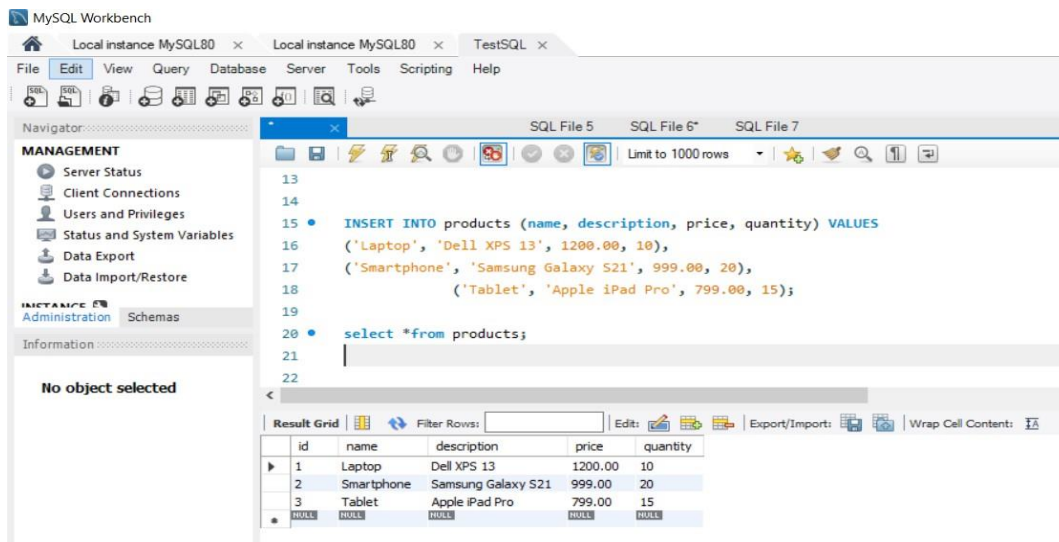
Action Output

#	Time	Action	Message
✓ 1	10:03:42	CREATE DATABASE kluh	1 row(s) affected
✓ 2	10:04:00	show databases	10 row(s) returned
✓ 3	10:16:48	USE kluh	0 row(s) affected

Create Table

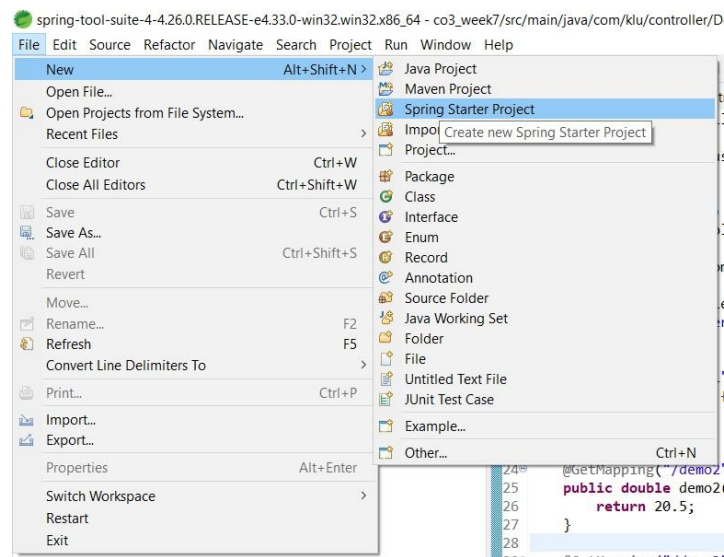


Insert Sample Data



Project creation:

Step 1:



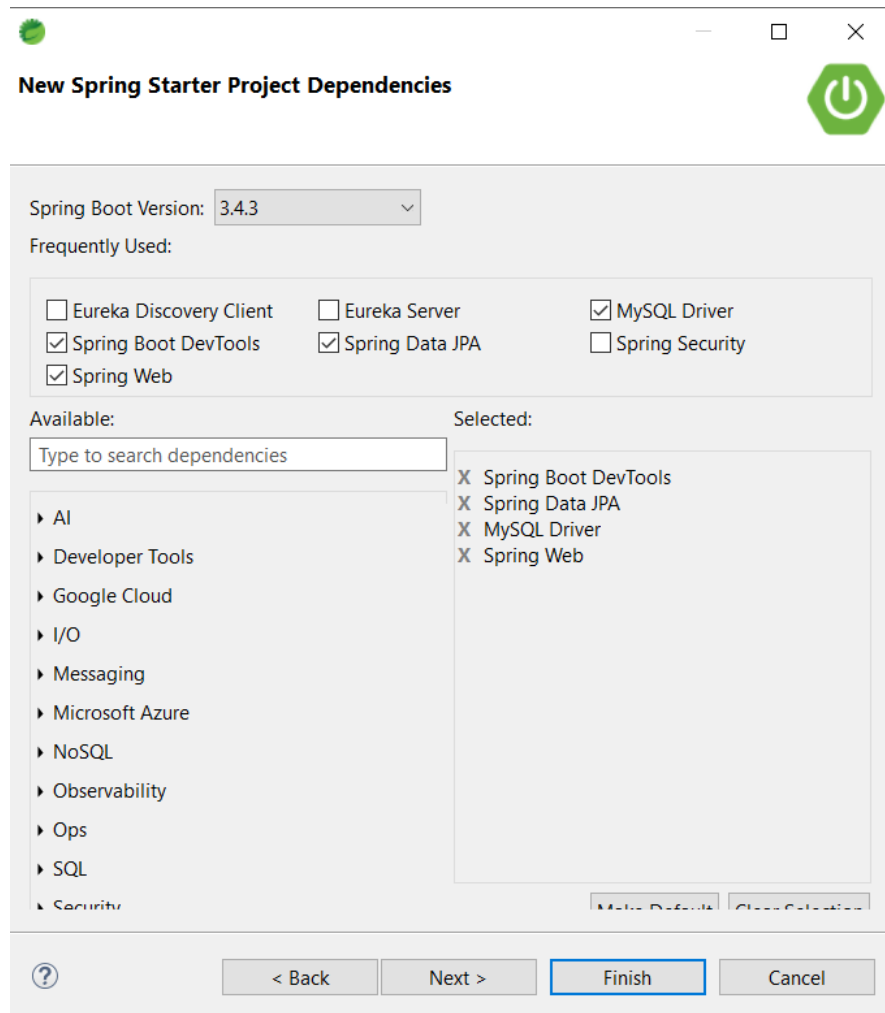
Step2

The 'New Spring Starter Project' dialog box is shown. It contains the following fields and options:

- Service URL: <https://start.spring.io>
- Name: `week8_co3`
- ☒ Use default location
- Location: `C:\files\springboot\spring-tool-suite-4-4.26.0.RELEASE-e4.33.0-win3` (with a 'Browse' button)
- Type: `Maven` (dropdown)
- Packaging: `Jar` (dropdown)
- Java Version: `17` (dropdown)
- Language: `Java` (dropdown)
- Group: `com.klu`
- Artifact: `week8_co3`
- Version: `0.0.1-SNAPSHOT`
- Description: `Demo project for Spring Boot`
- Package: `com.klu`
- Working sets section:
 - ☐ Add project to working sets (with a 'New...' button)
 - Working sets: (dropdown menu) (with a 'Select...' button)

At the bottom, there are navigation buttons: '< Back', 'Next >', 'Finish', and 'Cancel'.

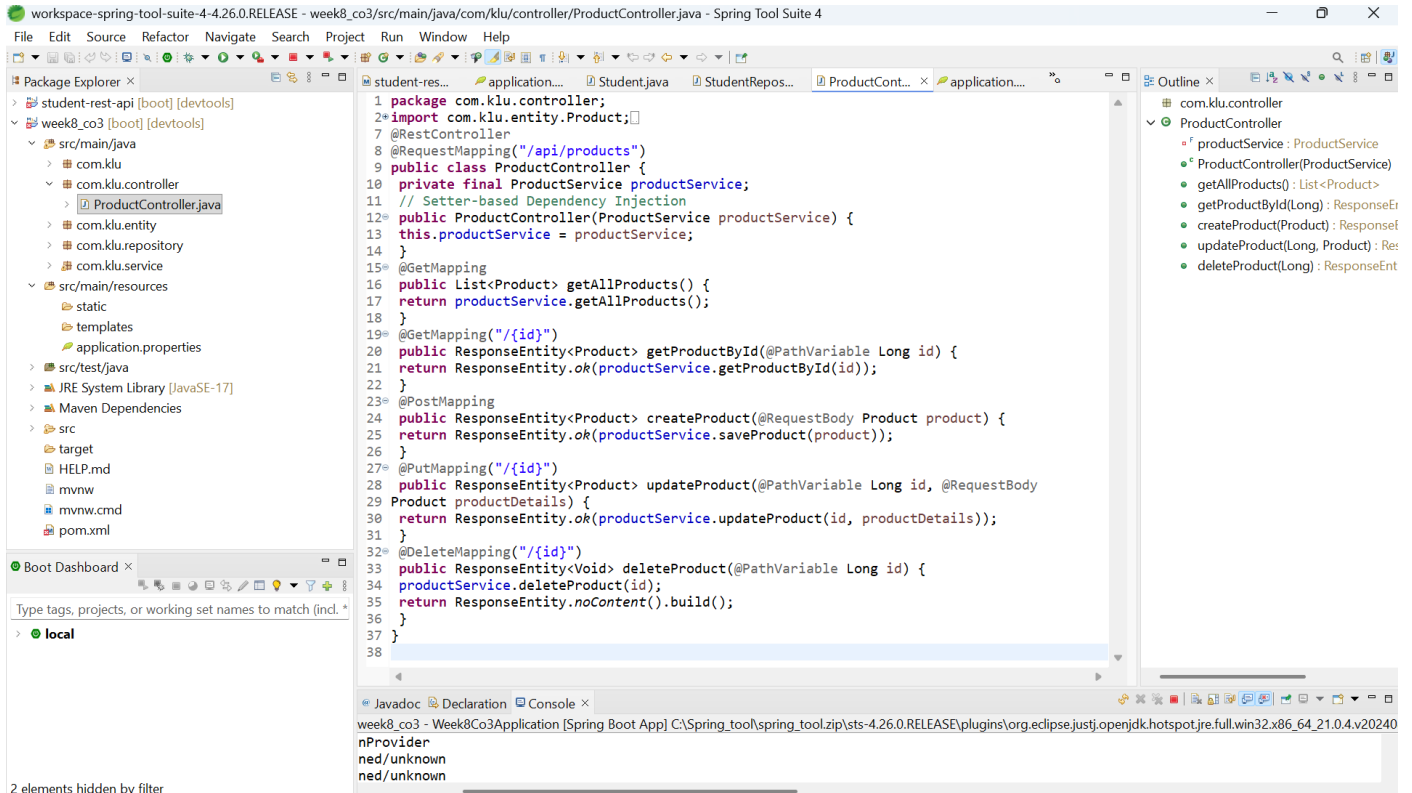
Step 3



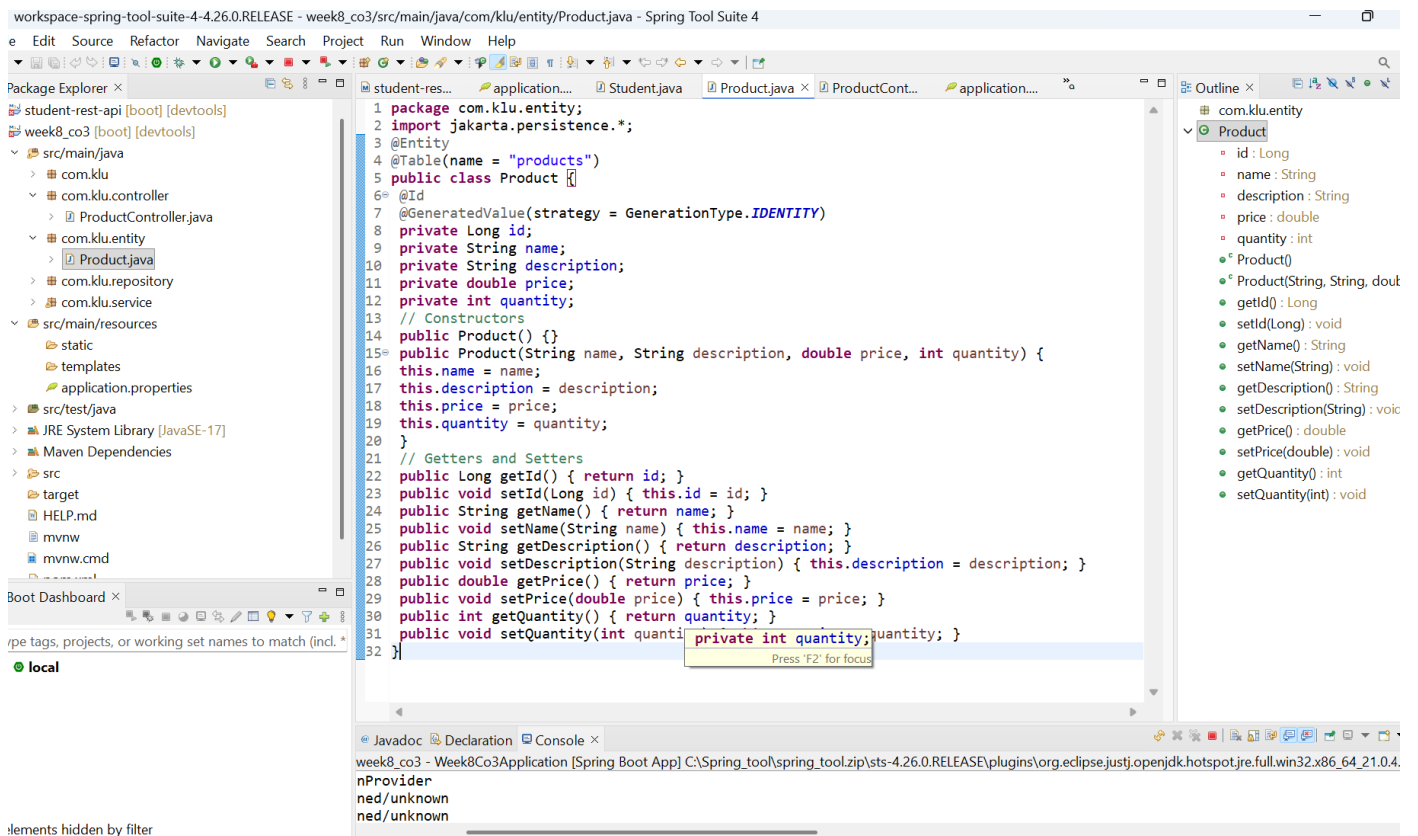
```
week8_co3 [boot] [devtools]
├── src/main/java
│   ├── com.klu
│   │   ├── com.klu.controller
│   │   │   └── ProductController.java
│   │   ├── com.klu.entity
│   │   │   └── Product.java
│   │   ├── com.klu.repository
│   │   │   └── ProductRepository.java
│   │   ├── com.klu.service
│   │   │   ├── ProductService.java
│   │   │   └── ProductServiceImpl.java
│   └── src/main/resources
│       ├── static
│       ├── templates
│       └── application.properties
```

Programs:

ProductController.java



Product.java:



ProductRepository.java

workspace-spring-tool-suite-4-4.26.0.RELEASE - week8_co3/src/main/java/com/klu/repository/ProductRepository.java - Spring Tool Suite 4

File Edit Source Refactor Navigate Search Project Run Window Help

Package Explorer ×

- student-rest-api [boot] [devtools]
- week8_co3 [boot] [devtools]
 - src/main/java
 - com.klu
 - com.klu.controller
 - ProductController.java
 - com.klu.entity
 - Product.java
 - com.klu.repository
 - ProductRepository.java**
 - com.klu.service
 - src/main/resources
 - static
 - templates
 - application.properties
 - src/test/java
 - JRE System Library [JavaSE-17]
 - Maven Dependencies
 - src
 - target
 - HELP.md
 - mvnw

Boot Dashboard ×

Type tags, projects, or working set names to match (incl. *)

local

student-res... application... Product.java ProductRepos... ProductCont... application...

```
1 package com.klu.repository;
2 import com.klu.entity.Product;
3
4 @Repository
5 public interface ProductRepository extends JpaRepository<Product, Long> {
6 }
7
8
```

Outline ×

- com.klu
 - Product

ProductService.java

workspace-spring-tool-suite-4-4.26.0.RELEASE - week8_co3/src/main/java/com/klu/service/ProductService.java - Spring Tool Suite 4

Edit Source Refactor Navigate Search Project Run Window Help

Package Explorer ×

- student-rest-api [boot] [devtools]
- week8_co3 [boot] [devtools]
 - src/main/java
 - com.klu
 - com.klu.controller
 - ProductController.java
 - com.klu.entity
 - Product.java
 - com.klu.repository
 - ProductRepository.java
 - com.klu.service
 - ProductService.java**
 - ProductServiceImpl.java
 - src/main/resources
 - static
 - templates
 - application.properties
 - src/test/java
 - JRE System Library [JavaSE-17]
 - Maven Dependencies

student-res... Product.java ProductRepos... **ProductServ...** ProductCont... application...

```
1 package com.klu.service;
2 import com.klu.entity.Product;
3
4 public interface ProductService {
5     List<Product> getAllProducts();
6     Product getProductById(Long id);
7     Product saveProduct(Product product);
8     Product updateProduct(Long id, Product product);
9     void deleteProduct(Long id);
10 }
```

ProductServiceImpl.java

The screenshot displays the Eclipse IDE interface for a Spring Boot application. The main editor shows the `ProductServiceImpl.java` file, which implements the `ProductService` interface. The code includes a constructor for dependency injection and several methods to manage products.

```
package com.klu.service;

import com.klu.entity.Product;

@Service
public class ProductServiceImpl implements ProductService {
    private final ProductRepository productRepository;

    // Constructor-based Dependency Injection
    public ProductServiceImpl(ProductRepository productRepository) {
        this.productRepository = productRepository;
    }

    @Override
    public List<Product> getAllProducts() {
        return productRepository.findAll();
    }

    @Override
    public Product getProductById(Long id) {
        return productRepository.findById(id)
            .orElseThrow(() -> new RuntimeException("Product not found"));
    }

    @Override
    public Product saveProduct(Product product) {
        return productRepository.save(product);
    }

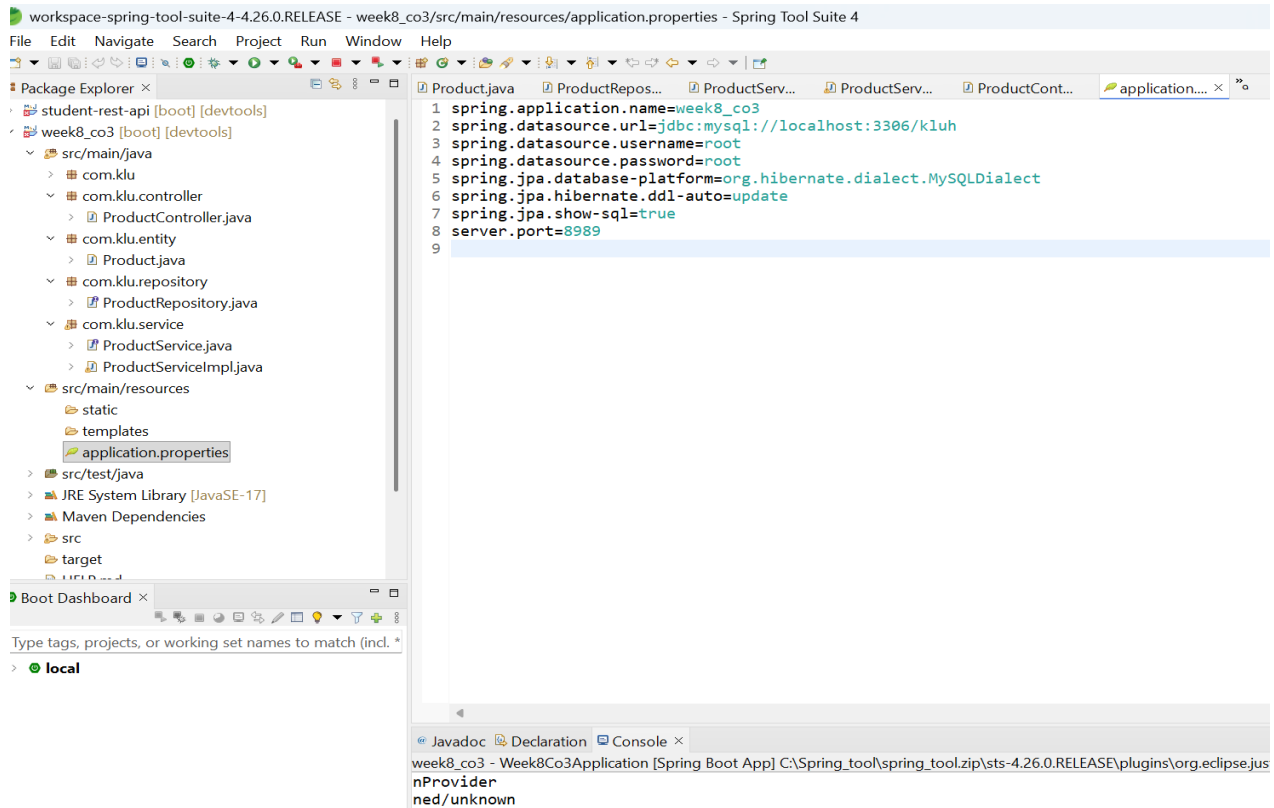
    @Override
    public Product updateProduct(Long id, Product productDetails) {
        Product existingProduct = getProductById(id);
        existingProduct.setName(productDetails.getName());
        existingProduct.setDescription(productDetails.getDescription());
        existingProduct.setPrice(productDetails.getPrice());
        existingProduct.setQuantity(productDetails.getQuantity());
        return productRepository.save(existingProduct);
    }

    @Override
    public void deleteProduct(Long id) {
        productRepository.deleteById(id);
    }
}
```

The Package Explorer on the left shows the project structure, including the `com.klu` package and its sub-packages. The Outline on the right shows the class hierarchy, including `ProductServiceImpl` and its methods.

At the bottom, the Javadoc Declaration and Console tabs are visible. The Console shows the output of the application, including the message "nProvider ned/unknown ned/unknown".

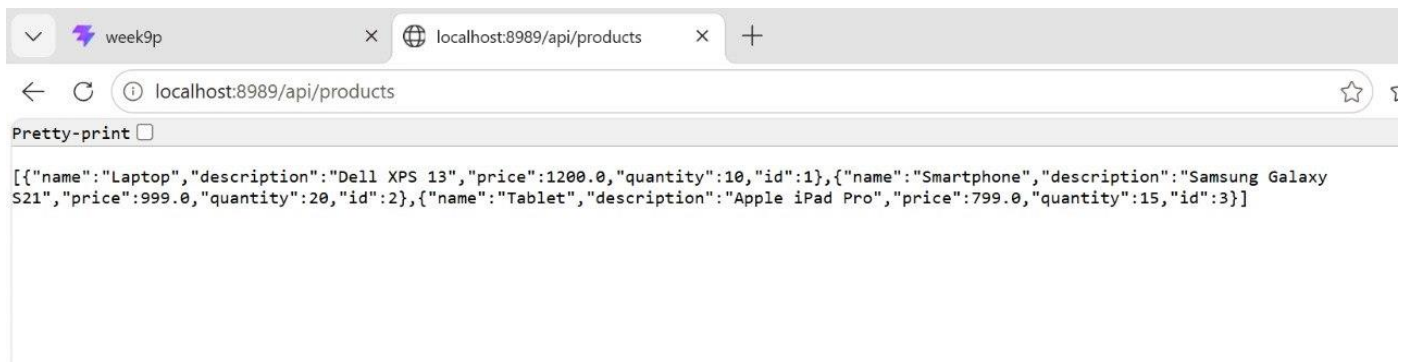
Application.properties:



API Endpoints

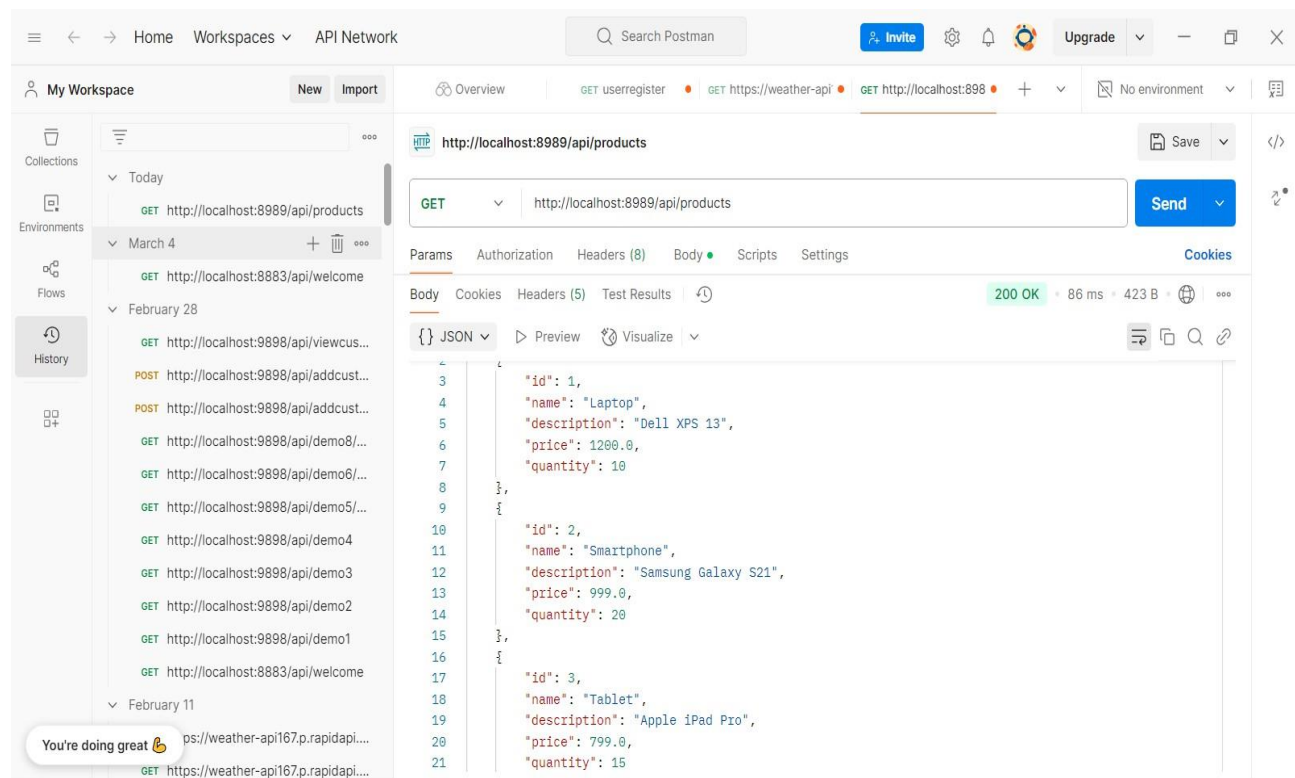
HTTP Method	Endpoint	Description
GET	http://localhost:8989/api/products	Get all products
GET	http://localhost:8989/api/products/{id}	Get product by ID
POST	http://localhost:8989/api/products	Create new product
HTTP Method	Endpoint	Description
PUT	http://localhost:8989/api/products/{id}	Update existing product
DELETE	http://localhost:8989/api/products/{id}	Delete a product

Outputs:



Postman testing:

1. Using get method



Using post method:

The screenshot shows a REST client interface with a POST request to `http://localhost:8989/api/products`. The request body is a JSON object: `{ "name": "WifiRouter", "description": "dlink", "price": 11200.00, "quantity": 98 }`. The response is `200 OK` with a JSON body: `{ "name": "WifiRouter", "description": "dlink", "price": 11200.0, "quantity": 98, "id": 4 }`.

After post method ,Database table updated :

```
19
20 • select *from products;
21
```

	id	name	description	price	quantity
▶	1	wifirouter	dlink	11200	98
	2	Smartphone	Samsung Galaxy S21	999	20
	3	Tablet	Apple iPad Pro	799	15
•	NULL	NULL	NULL	NULL	NULL

POST Get data

GET http://localhost:8081/st

No environment

My Collection > Get data

POST http://localhost:8989/api/products

Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

raw JSON Schema Beautify

```
1 {
2   "name": "WifiRouter",
3   "description": "dlink",
4   "price": 11200.00,
5   "quantity": 98
6 }
```

Body 200 OK • 263 ms • 242 B • Save Response

JSON Preview Visualize

```
1 {
2   "name": "WifiRouter",
3   "description": "dlink",
4   "price": 11200.0,
5   "quantity": 98,
6   "id": 4
7 }
```

Fix the status code assertion